

Laboratorio: pipeline OpenTelemetry end-to-end con Jaeger y Prometheus en GCP y AWS

MATERIA	ENTREGABLE
Observabilidad en ambientes productivos	Reporte técnico del laboratorio Octubre 2026

Estudiantes	
1	David Aníbal Vásquez
2	Sergio Andrés Cobos
3	Sebastián Bedoya Flórez

1. Introducción

Este laboratorio implementa un pipeline de observabilidad de extremo a extremo basado en OpenTelemetry (OTel) para una aplicación de dos microservicios: service-a, que recibe órdenes de compra, y service-b, que administra el inventario. Los dos dependen entre sí por HTTP y comparten una base de datos PostgreSQL. El objetivo es capturar los tres pilares de la observabilidad desde el código hasta los backends: métricas en Prometheus, logs estructurados y trazas distribuidas en Jaeger. Además, se busca que las tres señales se puedan correlacionar a partir de un mismo identificador, el `trace_id`, que viaja con cada request según el estándar W3C Trace Context (W3C, 2021).

El trabajo se organizó en las cuatro fases del enunciado: instrumentación con el SDK, despliegue del OpenTelemetry Collector, backends y visualización, y análisis del costo de la instrumentación (overhead). El pipeline se ejecutó en un entorno local con Docker y se desplegó en Google Cloud sobre GKE. La infraestructura para AWS (ECS Fargate) quedó definida y validada como código, pero no se desplegó, por las razones que se explican en la sección 7.

Tabla 1

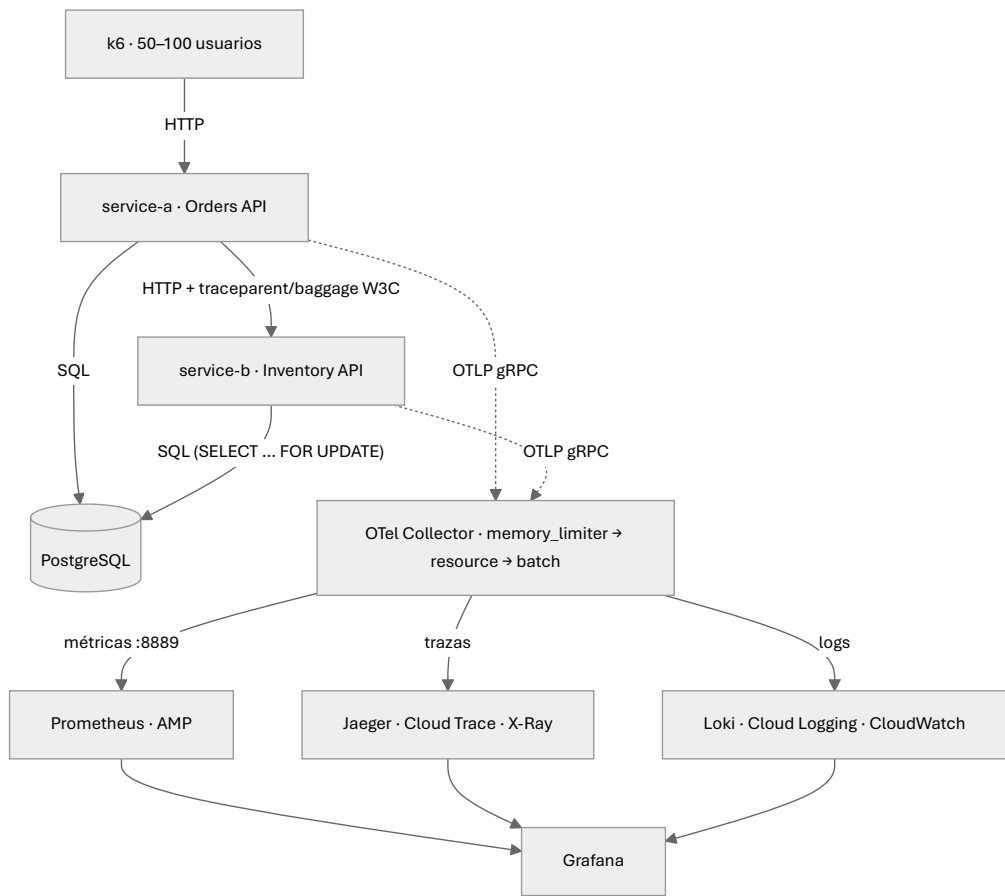
Datos generales del laboratorio

Dimensión	Dato
Aplicación	service-a (Orders API) → service-b (Inventory API), Python 3.12, FastAPI, SQLAlchemy, PostgreSQL 17
Señales	Trazas (OTLP → Jaeger / Cloud Trace / X-Ray), métricas (OTLP → Prometheus), logs JSON con <code>trace_id</code> (OTLP → Loki / Cloud Logging / CloudWatch)
Entornos	Local (docker compose, Colima), GCP (GKE Standard, 2 nodos e2-standard-4), AWS (ECS Fargate, definido en Terraform)
Versiones	OTel Python SDK 1.45 · OTel Collector contrib 0.162 · Jaeger 2.20 · Prometheus 3.13 · Loki 3.7 · Grafana 13 · k6 2.3
Repositorio	github.com/sergioandresco/Final-Project-Software-Architecture , carpeta <code>OpenTelemetry/</code>

2. Arquitectura de la solución

La arquitectura separa con claridad tres responsabilidades. Los servicios **generan** la telemetría mediante el SDK de OpenTelemetry. El Collector la **recibe, procesa y enruta**. Los backends la **almacenan y la muestran**. Los servicios solo conocen el protocolo OTLP y la dirección del Collector; qué backend recibe cada señal es una decisión de configuración del Collector, distinta para cada entorno. Por eso el mismo código funciona sin cambios en local, en GCP y en AWS.

Figura 1
Arquitectura del pipeline de observabilidad



Nota. Elaboración propia. Las líneas punteadas indican envío de telemetría por OTLP; las continuas, tráfico de la aplicación.

Una orden (POST /api/orders) produce una traza de unos 20 spans que cruza ambos servicios (tabla 2). Una parte de esos spans la crea la auto-instrumentación (HTTP servidor, HTTP cliente y SQL) y otra la crean los spans propios que se agregaron alrededor de la lógica de negocio.

Tabla 2
Recorrido de una orden y spans que genera

Paso	Servicio	Span	Origen
Recepción de la orden	service-a	POST /api/orders	Automático (FastAPI)
Validación	service-a	orders.validate	Propio
Reserva de inventario	service-a	orders.reserve_inventory → POST (cliente)	Propio + automático (httpx), que inyecta traceparent
Reserva en inventario	service-b	POST /api/products/{id}/reserve	Automático; extrae traceparent y continúa la misma traza
Lógica de reserva	service-b	inventory.reserve_stock	Propio
Acceso a datos	service-b	SELECT ... FOR UPDATE, UPDATE, INSERT	Automático (SQLAlchemy)
Cálculo del total	service-a	orders.calculate_total	Propio
Persistencia de la orden	service-a	INSERT orders	Automático (SQLAlchemy)

3. Fase 1: instrumentación con el SDK de OpenTelemetry

La configuración común de ambos servicios vive en el módulo `services/shared/telemetry.py`. Se eligió **instrumentación programática** en lugar del agente automático (`opentelemetry-instrument`) por dos razones. Primero, permite apagar toda la telemetría con `OTEL_SDK_DISABLED=true` sin cambiar la imagen, que es el modo de comparación del benchmark. Segundo, permite activar o desactivar cada señal por separado con las variables estándar `OTEL_{TRACES,METRICS,LOGS}_EXPORTER=none`, que se usaron en el análisis por componente.

La **auto-instrumentación** cubre los tres puntos de entrada y salida de cada servicio: el servidor HTTP (FastAPI), el cliente HTTP (`httpx`) y la base de datos (SQLAlchemy). Sobre ella se agregaron **spans propios** solo en la lógica de negocio crítica, con atributos que dan contexto al diagnóstico, como el número de orden, el total y el stock restante, y eventos como `inventory.insufficient_stock`. Para que los paneles de errores y las trazas con estado de error tuvieran datos reales, `service-b` incluye una **inyección de fallas** controlada: el 1% de las reservas responde 503 a propósito.

La **propagación de contexto** usa W3C Trace Context y Baggage. El encabezado `traceparent` lleva el identificador de la traza de `service-a` a `service-b`, y el `baggage` transporta el identificador del cliente, que `service-b` agrega como atributo de su span. Los **logs** se escriben en JSON en la salida estándar con el `trace_id` y el `span_id` del span activo, y en paralelo se envían por OTLP. Así, la misma línea de log se puede encontrar por su identificador de traza en cualquier backend.

Tabla 3

Señales emitidas por los servicios

Señal	Qué se emite	Cómo
Trazas	Spans HTTP, SQL y de negocio con atributos semánticos	TracerProvider + BatchSpanProcessor → OTLP/gRPC
Métricas	<code>http.server.request.duration</code> , <code>http.server.active_requests</code> , métricas de negocio (<code>orders.created</code> , <code>inventory.reservations</code>) y CPU/memoria del proceso	MeterProvider + PeriodicExportingMetricReader (10 s) → OTLP/gRPC
Logs	JSON en stdout con <code>trace_id/span_id</code> , y el mismo registro por OTLP	JsonFormatter + LoggingHandler → OTLP/gRPC
Contexto	<code>traceparent</code> , <code>tracestate</code> y <code>baggage</code> entre servicios	Propagadores W3C TraceContext + Baggage

Durante las pruebas aparecieron dos ajustes. FastAPI, desde su versión 0.140, trae una telemetría propia que se activa sola al detectar los providers globales; se desactivó explícitamente para que la única fuente de spans fuera la instrumentación contrib. Además, las sondas de salud de Kubernetes consultaban la base de datos cada 10 segundos y generaban trazas sin valor de negocio; esa consulta se ejecuta ahora con la instrumentación suprimida.

4. Fase 2: despliegue del OpenTelemetry Collector

El Collector funciona como **gateway**: recibe la telemetría de todos los servicios, la procesa y la envía a los backends. Tener este intermediario saca de la aplicación el agrupamiento, los reintentos y la conversión de formatos, y permite cambiar de backend sin redespregar los servicios. Hay una configuración por entorno: `otel-collector-local.yaml`, `otel-collector-gcp.yaml` y `otel-collector-aws.yaml`. Las tres se validaron con `otelcol-contrib validate`.

El orden de los procesadores sigue la recomendación del proyecto (tabla 4). `memory_limiter` va primero, para rechazar datos antes de que el Collector se quede sin memoria. `resource` agrega atributos comunes a las tres señales. `batch` va al final, para agrupar los envíos y reducir las llamadas de red.

Tabla 4

Pipeline del Collector

Componente	Configuración	Propósito
Receiver otlp	gRPC :4317 y HTTP :4318	Recibir trazas, métricas y logs de los servicios
Processor memory_limiter	400 MiB / pico 100 MiB (local); 80% / 20% (nube)	Aplicar <i>back-pressure</i> antes de un desbordamiento de memoria
Processor resource_detection	Detectores gcp o ecs (solo nube)	Agregar región, clúster y tarea automáticamente
Processor resource	deployment.environment, collector.pipeline	Enriquecer las tres señales con datos del entorno
Processor batch	Lotes de 1024–2048 elementos o 2 s	Reducir llamadas a los backends
Telemetría interna	Prometheus :8888, nivel detailed	Medir el propio Collector (CPU, datos rechazados, colas)

Tabla 5

Destino de cada señal según el entorno

Señal	Local	GCP (GKE)	AWS (ECS Fargate)
Trazas	Jaeger	Jaeger + Cloud Trace	AWS X-Ray
Métricas	Endpoint Prometheus :8889	Endpoint :8889 + ServiceMonitor	Amazon Managed Prometheus (SigV4) + :8889
Logs	Loki	Cloud Logging + Loki	CloudWatch Logs
Credenciales	No aplica	Workload Identity, sin llaves	Rol IAM de la tarea

En GCP el Collector se despliega en GKE con un chart de Helm y se autentica con **Workload Identity**: la cuenta de servicio de Kubernetes actúa como una cuenta de servicio de Google con permisos solo de escritura en Cloud Logging y Cloud Trace, así que no hay llaves JSON montadas en el pod. En AWS se definió como un servicio Fargate cuya configuración viaja en una variable de entorno (`--config=env:OTEL_COLLECTOR_CONFIG`), con un rol IAM limitado a X-Ray, CloudWatch Logs y escritura en Amazon Managed Prometheus.

5. Fase 3: backends, visualización y correlación

5.1 Métricas y dashboard

Prometheus funciona como **base de datos de métricas**: cada cinco segundos lee el endpoint que expone el Collector y guarda las series. Grafana no almacena métricas; las consulta en Prometheus y las grafica. El dashboard tiene los seis paneles que pide el enunciado: cuatro indicadores de nivel de servicio (SLI) que siguen las señales doradas (Beyer et al., 2016), el consumo de CPU y los errores del propio Collector (tabla 6).

Tabla 6

Paneles del dashboard de Grafana

Panel	Pregunta que responde	Consulta (resumen)
SLI 1 · Throughput	¿Cuánto tráfico atiende cada servicio?	<code>rate(http_server_request_duration_seconds_count[1m])</code>
SLI 2 · Tasa de errores 5xx	¿Qué porcentaje de requests falla?	errores 5xx / total
SLI 3 · Latencia p50/p95/p99	¿Qué tan rápido responde, incluso en el peor 1%?	histogram_quantile sobre el histograma de duración
SLI 4 · Disponibilidad	¿Se cumple el objetivo de 99% de requests sin error?	1 – errores / total (ventana de 5 min)
CPU	¿Cuánto procesador usan los servicios y el Collector?	<code>rate(process_cpu_time_seconds_total[1m])</code>
Errores del Collector	¿Se está perdiendo telemetría?	<code>otelcol_exporter_send_failed_*</code> , <code>otelcol_receiver_refused_*</code>

5.2 Correlación entre señales con el trace_id

La correlación es lo que convierte tres herramientas separadas en un sistema de observabilidad. Grafana se configuró para saltar entre señales usando el `trace_id` como pivote, siguiendo el modelo de integración de trazas de Grafana Labs (s. f.). Desde una línea de log en Loki, el enlace "Ver traza en Jaeger" abre la traza completa de ese request. Desde un span en Jaeger, el enlace inverso muestra los logs con el mismo identificador. En la prueba, una orden produjo una traza de 20 spans, y la búsqueda de su `trace_id` en Loki devolvió exactamente las dos líneas esperadas: "Orden creada" de service-a y "Stock reservado" de service-b. En GCP la misma correlación es nativa: cada entrada de Cloud Logging trae un campo `trace` que enlaza con su traza en Cloud Trace.

Tabla 7

Mecanismos de correlación verificados

Dirección	Mecanismo	Resultado
Log → traza	Campo derivado de Loki sobre el metadato <code>trace_id</code>	Abre la traza en Jaeger dentro de Grafana
Traza → logs	<code>tracesToLogsV2</code> del datasource de Jaeger	Filtra Loki por el <code>trace_id</code> del span
Traza → métricas	<code>tracesToMetrics</code>	Muestra la latencia p99 del servicio del span
Respuesta → traza	service-a devuelve el encabezado <code>X-Trace-Id</code>	El cliente puede citar la traza al reportar un problema

Dirección	Mecanismo	Resultado
Log en GCP → traza	Campo <code>trace</code> de Cloud Logging	Enlace directo a Cloud Trace

5.3 Despliegue en Google Cloud

La infraestructura de GCP se creó con Terraform: 19 recursos, entre ellos el clúster GKE, el registro de imágenes, las cuentas de servicio y la federación de identidad. Las imágenes se construyeron con Cloud Build, y la aplicación, el Collector y los backends se instalaron con Helm. La verificación de punta a punta se resume en la tabla 8.

Tabla 8

Verificación del despliegue en GKE

Verificación	Resultado
Infraestructura	Clúster <code>otel-lab</code> con 2 nodos (8 vCPU, 32 GB) y 19 recursos gestionados por Terraform
Imágenes	Construidas con Cloud Build (1 min 18 s) y publicadas en Artifact Registry
Pods	8/8 de la aplicación y 6/6 de monitoreo en estado Running
API pública	<code>POST /api/orders</code> por el balanceador de carga responde 201 con su <code>trace_id</code>
Trazas	La misma traza (20 spans, 2 servicios) en Jaeger y en Cloud Trace
Logs	Cloud Logging con el campo <code>trace</code> vinculado a Cloud Trace
Métricas	ServiceMonitor del Collector en estado UP; dashboard con datos de GKE
Carga (30 usuarios × 15 min)	61.4 req/s, p95 73.8 ms, p99 120.3 ms y 0.63% de errores (la falla inyectada del 1%)

6. Fase 4: análisis de overhead

6.1 Metodología

El benchmark se ejecutó en un portátil Apple M1 (8 núcleos, 8 GB) con Colima (6 vCPU, 8 GB). Cada servicio estuvo limitado a 1 CPU y 512 MiB para que las corridas fueran comparables. Se hicieron dos experimentos.

El primero, **experimento A**, cumple el requisito del enunciado: k6 con 100 usuarios concurrentes durante 5 minutos, pausas de 200 a 800 ms entre acciones y una mezcla de 70% de órdenes, 20% de consultas al catálogo y 10% de consultas de órdenes. Cada caso arrancó con el stack recién creado y un calentamiento de 30 segundos. Se comparó la aplicación sin instrumentación, con instrumentación y muestreo del 100%, y con instrumentación y muestreo del 10%. El segundo, **experimento B**, es una ablación: cada señal se activó por separado con una tasa fija de unas 90 solicitudes por segundo, por debajo de la saturación, para medir el **costo de CPU por solicitud** de cada componente.

6.2 Resultados

Tabla 9

Experimento A: 100 usuarios concurrentes durante 5 minutos

Métrica	Sin OTel	OTel 100%	Variación	OTel 10%	Variación
Latencia p50 (ms)	7.30	123.67	+1593%	157.62	+2058%
Latencia p95 (ms)	146.35	751.14	+413%	1216.76	+731%
Latencia p99 (ms)	445.90	1263.60	+818 ms (+183%)	3587.06	+704%
Throughput (req/s)	210.32	151.42	-28.0%	125.53	-40.3%
CPU service-a (% de 1 núcleo)	53.4	96.9	+81%	98.3	+84%
CPU service-b (% de 1 núcleo)	35.8	64.9	+81%	62.8	+75%
Memoria service-a (MiB)	73.2	95.4	+30%	93.2	+27%
Memoria service-b (MiB)	64.5	83.5	+29%	80.9	+25%
OTel Collector (CPU / memoria)	No aplica	6.6% / 85 MiB		5.2% / 81 MiB	

Tabla 10

Experimento B: costo por componente a tasa fija (~90 req/s)

Variante	p50 (ms)	p99 (ms)	CPU por request en estado estable (ms)	Variación	Tiempo saturado
Sin OTel	4.6	60.2	3.15	No aplica	0%
Instrumentación sin providers (no-op)	6.2	2120.6	4.42	+1.27 ms (+40%)	16%
Solo métricas	5.5	380.3	4.18	+1.03 ms (+33%)	2%
Solo logs OTLP	5.4	90.4	4.03	+0.88 ms (+28%)	0%
Solo trazas (100%)	6.3	315.2	5.70	+2.55 ms (+81%)	2%
Completo (100%)	22.1	1718.7	8.21	+5.06 ms (+161%)	29-67%
Completo con muestreo 10%	7.1	937.0	5.89	+2.74 ms (+87%)	2%

Nota. CPU por request en estado estable = suma de las medianas de CPU de ambos servicios, sin contar los instantes saturados, dividida entre las solicitudes por segundo. Tiempo saturado = porcentaje de muestras con service-a al 90% o más de su núcleo. La variación entre corridas repetidas fue de unos ± 0.3 ms.

6.3 Análisis

Con 100 usuarios y pausas cortas, la aplicación sin instrumentar ya usa el 53% del núcleo de service-a. La instrumentación completa la lleva a la saturación (97%). Desde ese punto la latencia deja de reflejar el trabajo de cada solicitud y pasa a reflejar la **cola** de solicitudes en espera. Por eso el experimento A mide, más que nada, la **pérdida de capacidad**: con la misma infraestructura, el servicio instrumentado sostiene un 28% menos de tráfico. El caso con muestreo del 10% salió peor que el de 100%, en contra de lo esperado. El experimento B mostró que esa diferencia no se debe al muestreo, sino a episodios de saturación que aparecen de forma irregular entre corridas.

El experimento B separa el costo intrínseco de cada señal. La configuración completa agrega unos **5 ms de CPU por solicitud** (+161%) sumando los dos servicios. Las trazas son la señal más cara (+2.6 ms), porque cada orden genera unos 20 spans. Las siguen las métricas (+1.0 ms) y los logs (+0.9 ms). Una parte del costo corresponde a los envoltorios de instrumentación, que siguen activos aun sin SDK. La memoria crece entre 18 y 22 MiB por servicio, alrededor de 30%. El Collector, separado de la aplicación, consume entre 5% y 7% de un núcleo y unos 85 MiB.

El hallazgo más importante es la **inestabilidad metaestable**. Con todo activado, service-a corre estable a cerca de 42% de CPU, pero un pico transitorio genera cola. En Python, los endpoints síncronos comparten un grupo de 40 hilos que compiten por el GIL, así que con más cola cada solicitud cuesta más CPU, y el sistema no vuelve a su estado estable aunque la carga promedio no lo justifique. Bronson et al. (2021) describen este patrón como una falla metaestable. El servicio pasó entre 29% y 67% del tiempo saturado. Con muestreo head-based del 10%, el costo bajó a +2.7 ms por solicitud, la mediana de latencia volvió a 7 ms y el tiempo saturado cayó a 2%. Los logs y las métricas siguen emitiéndose para todas las solicitudes con su `trace_id`, así que la correlación se conserva para las trazas muestreadas. Es el mismo principio de muestreo que propuso Dapper para sostener el trazado distribuido a gran escala (Sigelman et al., 2010).

6.4 Recomendaciones

De los resultados se desprenden cuatro recomendaciones:

1. **Muestreo.** Usar muestreo head-based de 10% o menos en producción y, si se necesitan todas las trazas con error o lentas, agregar *tail sampling* en el Collector.
2. **Endpoints asíncronos o más réplicas.** Pasar los endpoints de entrada y salida a modo asíncrono (`httpx.AsyncClient`, `SQLAlchemy asyncio`) o escalar horizontalmente, para evitar la contención del grupo de hilos.
3. **Margen de capacidad.** Dejar un margen de capacidad del doble del CPU medido sin instrumentación al dimensionar servicios Python instrumentados.
4. **Granularidad de los spans SQL.** Revisar los spans SQL, que son la mayoría de cada traza, y filtrarlos o muestrearlos aparte si no aportan al diagnóstico.

7. Problemas encontrados y lecciones aprendidas

La implementación expuso varios problemas reales, propios de un ecosistema que cambia rápido (tabla 11). La mayoría no eran errores de código, sino diferencias entre versiones, o situaciones que solo aparecen bajo carga sostenida. Esto refuerza la idea de que el pipeline de observabilidad también debe observarse.

Tabla 11

Problemas encontrados, diagnóstico y solución

Problema	Diagnóstico	Solución
El datasource de Jaeger en Grafana responde 404	Jaeger 2.21 eliminó la API HTTP v1 que usa Grafana	Fijar Jaeger 2.20.0

Problema	Diagnóstico	Solución
El Jaeger de GKE perdía todas las trazas	Guardaba hasta 100.000 trazas en memoria y fue terminado por falta de memoria (OOMKilled)	Limitar a 20.000 trazas (max_traces)
Cloud Trace lleno de trazas sin negocio	Las sondas de salud de Kubernetes consultaban la base de datos cada 10 s	Suprimir la instrumentación en /health/ready (40 sondas → 0 spans)
Panel de errores del Collector con error de consulta	rate() sobre una expresión regular del nombre de métrica deja series duplicadas	Consultas explícitas por métrica
Opción del Collector marcada como obsoleta	Su reemplazo (resource_constant_labels) no agregaba etiquetas en la versión 0.162	Mantener la opción anterior y documentarlo
No aparecen exemplars en Prometheus	El SDK los genera, pero el exporter prometheus no los publica	Correlación métrica → traza desde Jaeger
Telemetría duplicada potencial en FastAPI	FastAPI 0.140+ trae telemetría propia que se autoconfigura	Desactivarla explícitamente
Incompatibilidad de versiones	La instrumentación de SQLAlchemy no soporta la versión 2.1	Fijar SQLAlchemy 2.0.54
Componentes renombrados en el Collector	otlp → otlp_grpc, otlphttp → otlp_http	Usar los nombres nuevos y validar la configuración

En cuanto a limitaciones, el despliegue en **AWS no se ejecutó**: la única cuenta disponible es corporativa y no se usó para fines académicos. El módulo de Terraform para ECS Fargate (red, balanceador, descubrimiento de servicios, cuatro servicios, CloudWatch Logs, Amazon Managed Prometheus e IAM) pasa terraform validate, y la configuración del Collector para AWS pasa otlcol-contrib validate, así que queda listo para ejecutarse en una cuenta personal o académica. Además, el benchmark se hizo en un portátil donde la carga y los servicios comparten la máquina, lo que introduce ruido entre corridas. Por eso el análisis por componente usa tasa fija y medianas de estado estable.

8. Conclusiones

El pipeline cumple los objetivos del laboratorio. Los dos servicios emiten trazas, métricas y logs con OpenTelemetry. El Collector desacopla la aplicación de los backends con una configuración por entorno, el contexto W3C se propaga entre servicios, y el trace_id funciona como pivote entre las tres señales, tanto en Grafana como en las herramientas nativas de GCP.

El análisis de overhead deja una lección que va más allá de la herramienta. La observabilidad completa tiene un costo medible: unos 5 ms de CPU por solicitud y 30% más de memoria en servicios Python. Con muestreo del 100%, además, reduce el margen de capacidad hasta volver frágil al servicio bajo carga. El muestreo y el dimensionamiento son, por lo tanto, decisiones de diseño tan importantes como la instrumentación misma. Con muestreo del 10% se conserva la correlación entre señales y el costo se reduce casi a la mitad.

Referencias

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media. <https://sre.google/sre-book/table-of-contents/>

Bronson, N., Aghayev, A., Charapko, A., & Zhu, T. (2021). Metastable failures in distributed systems. En *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '21)* (pp. 221–227). ACM. <https://doi.org/10.1145/3458336.3465286>

Grafana Labs. (s. f.). *Trace integration in Explore*. Grafana documentation. <https://grafana.com/docs/grafana/latest/explore/trace-integration/>

Grafana Labs. (s. f.). *Grafana k6 documentation*. <https://k6.io/docs/>

Jaeger Authors. (s. f.). *Architecture*. Jaeger documentation. <https://www.jaegertracing.io/docs/architecture/>

OpenTelemetry Authors. (s. f.). *OpenTelemetry Python*. <https://opentelemetry-python.readthedocs.io/>

OpenTelemetry Authors. (s. f.). *Collector*. OpenTelemetry documentation. <https://opentelemetry.io/docs/collector/>

Prometheus Authors. (s. f.). *Overview*. Prometheus documentation. <https://prometheus.io/docs/introduction/overview/>

Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., & Shanbhag, C. (2010). *Dapper, a large-scale distributed systems tracing infrastructure* (Google Technical Report dapper-2010-1). Google.

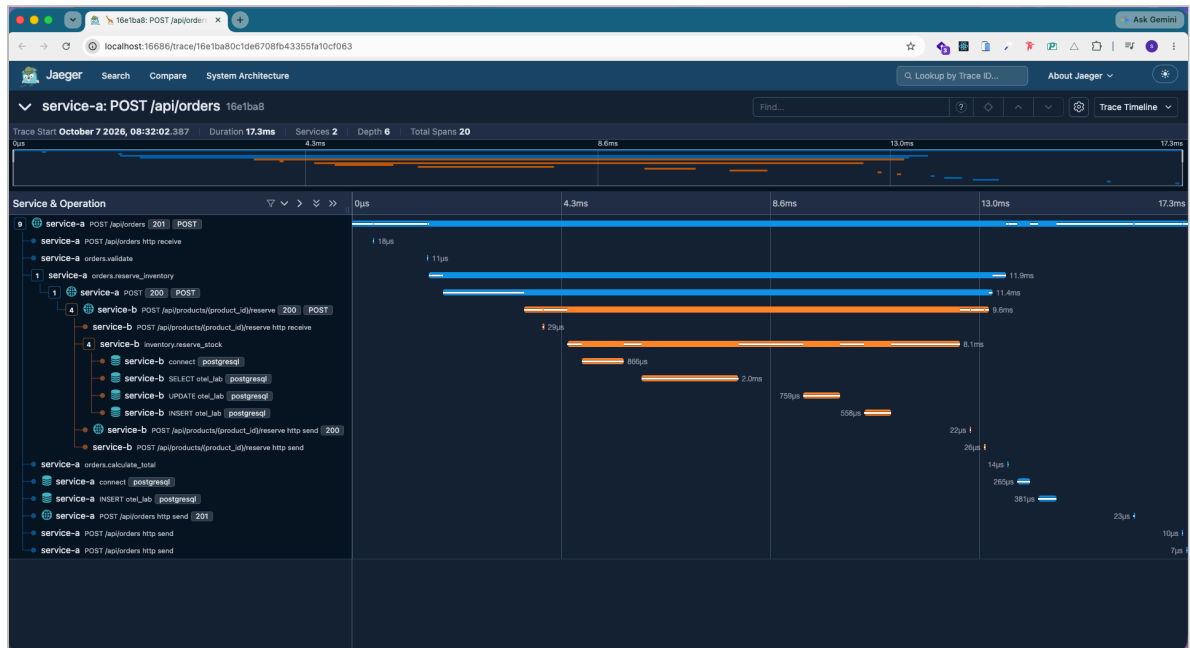
W3C. (2021). *Trace Context* (W3C Recommendation). <https://www.w3.org/TR/trace-context/>

Anexo A. Evidencias

Entorno local

Figura 2

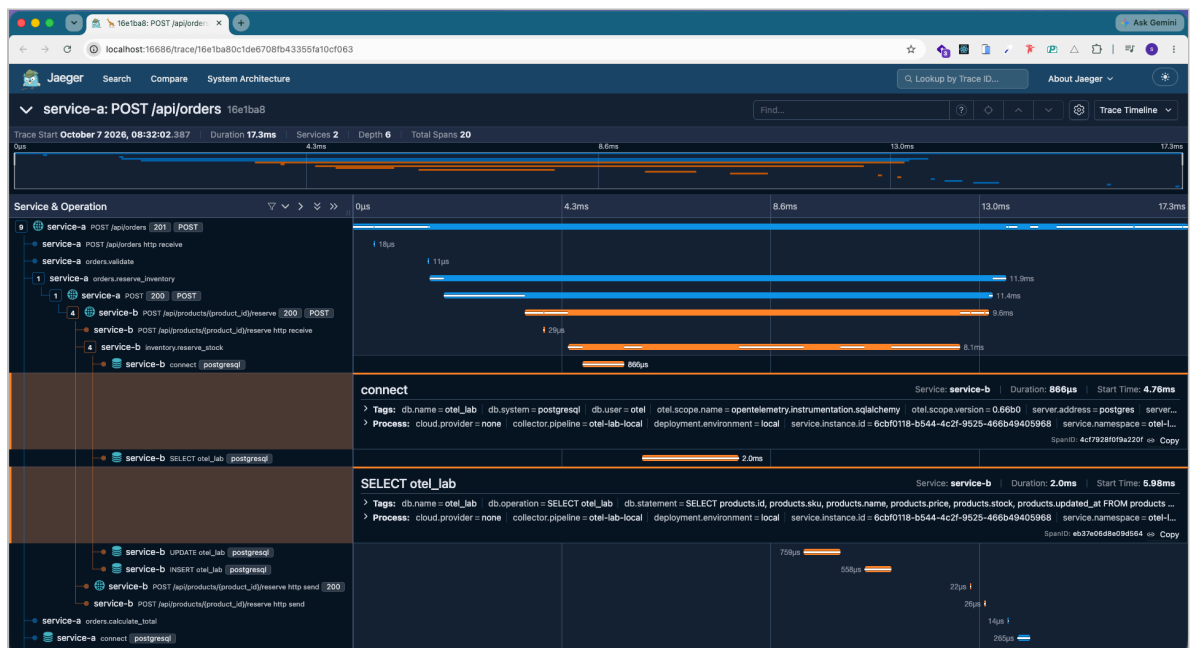
Traza completa de una orden en Jaeger



Nota. 20 spans en 2 servicios y 6 niveles de profundidad: spans HTTP automáticos, spans propios (orders.*, inventory.reserve_stock) y consultas SQL.

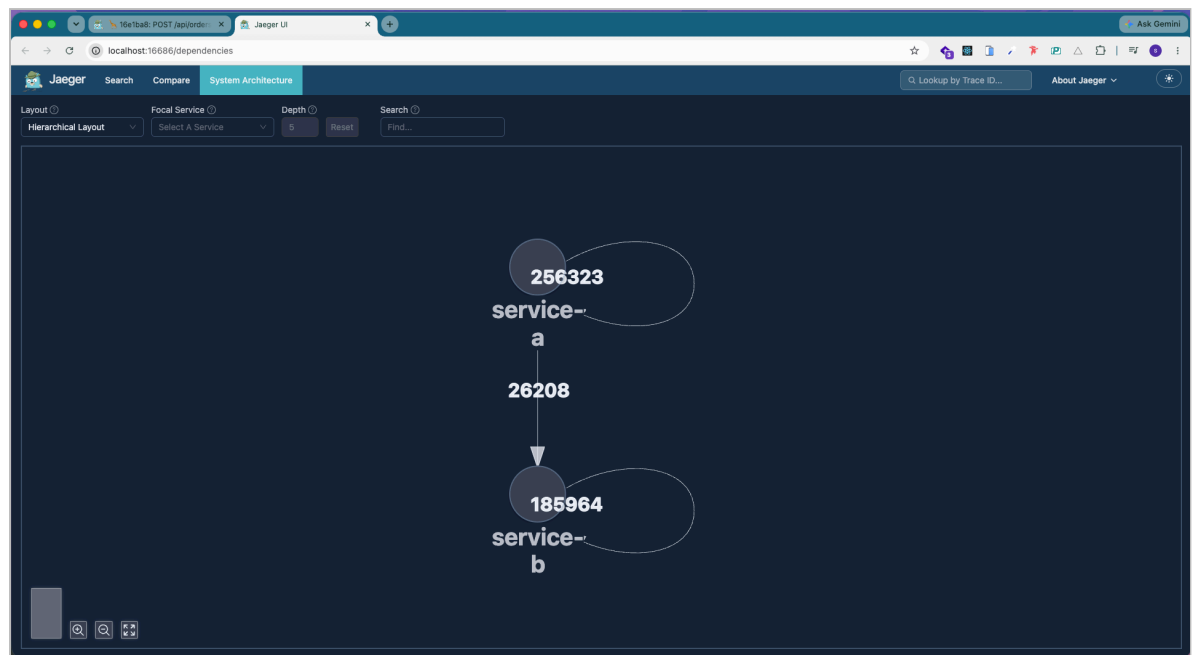
Figura 3

Detalle de los spans de base de datos



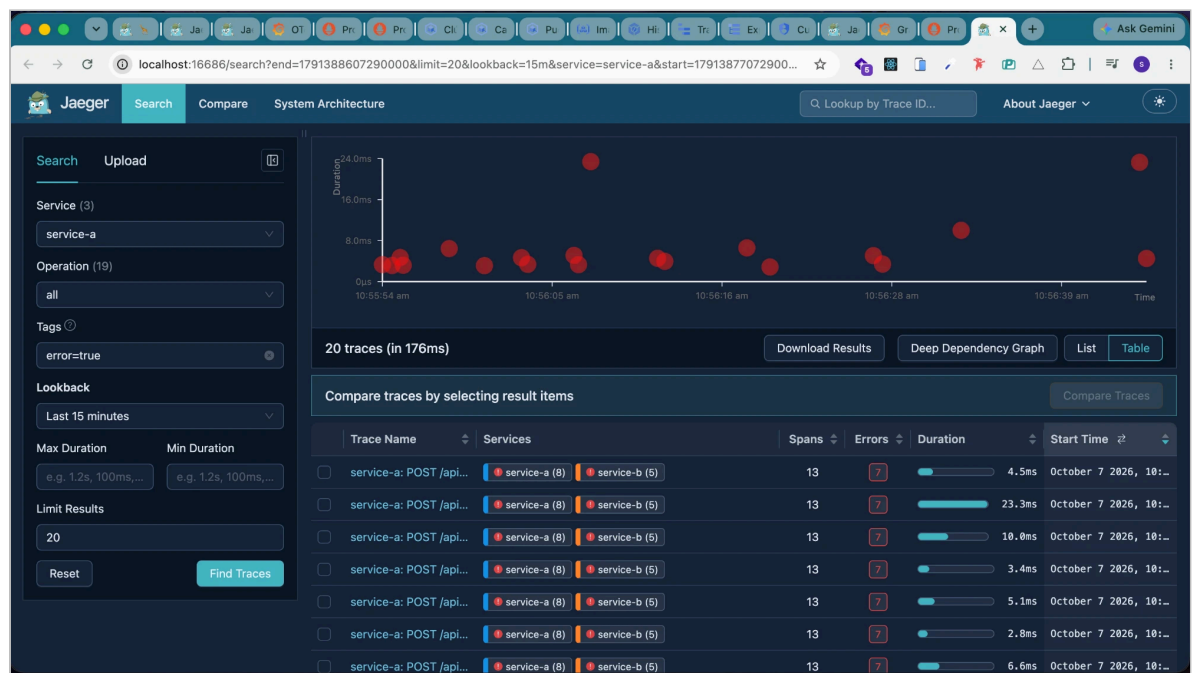
Nota. Atributos semánticos de la auto-instrumentación (db.system, db.statement) y del Resource agregado por el Collector.

Figura 4
Grafo de dependencias entre servicios



Nota. Jaeger construye la relación service-a → service-b a partir de la propagación del contexto W3C.

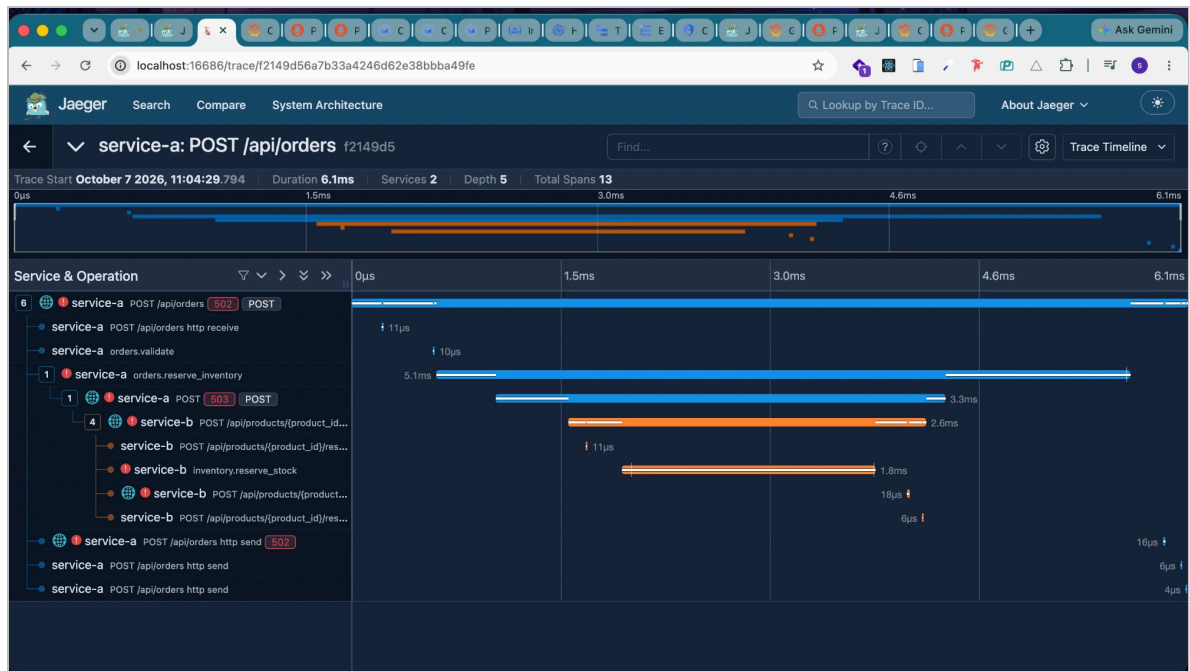
Figura 5
Búsqueda de trazas con error



Nota. Filtro error=true sobre las trazas afectadas por la falla inyectada del 1%.

Figura 6

Traza con error propagado entre servicios



Nota. service-b responde 503 en la reserva y service-a lo traduce a 502.

Figura 7

Dashboard de indicadores de nivel de servicio bajo carga

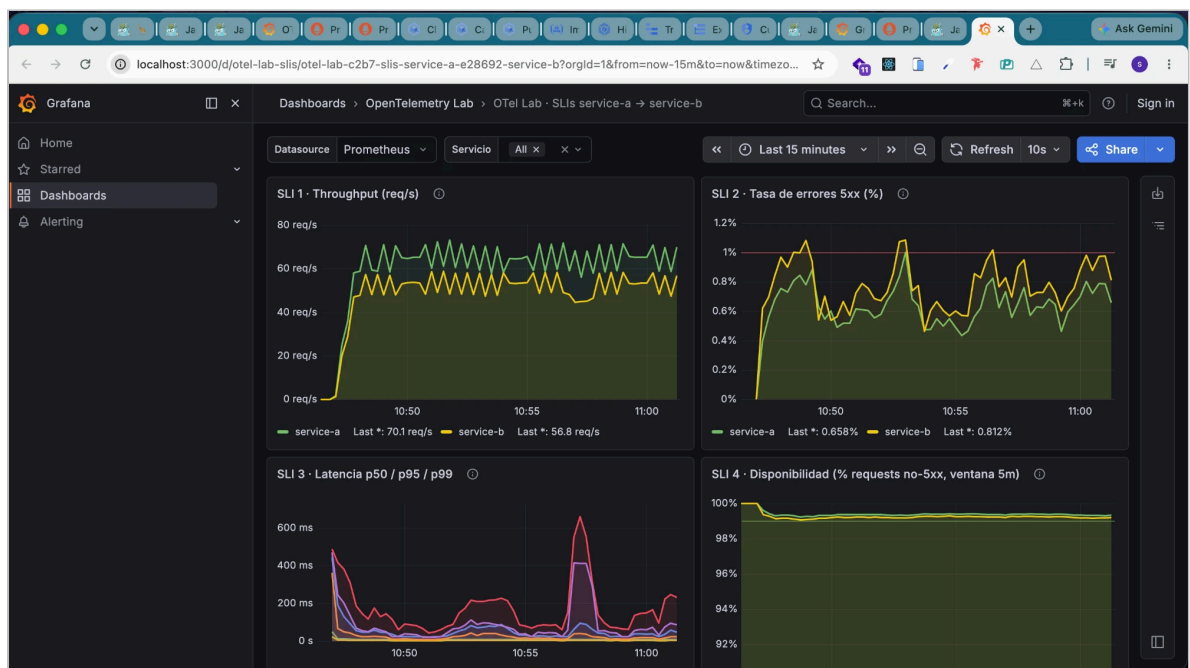
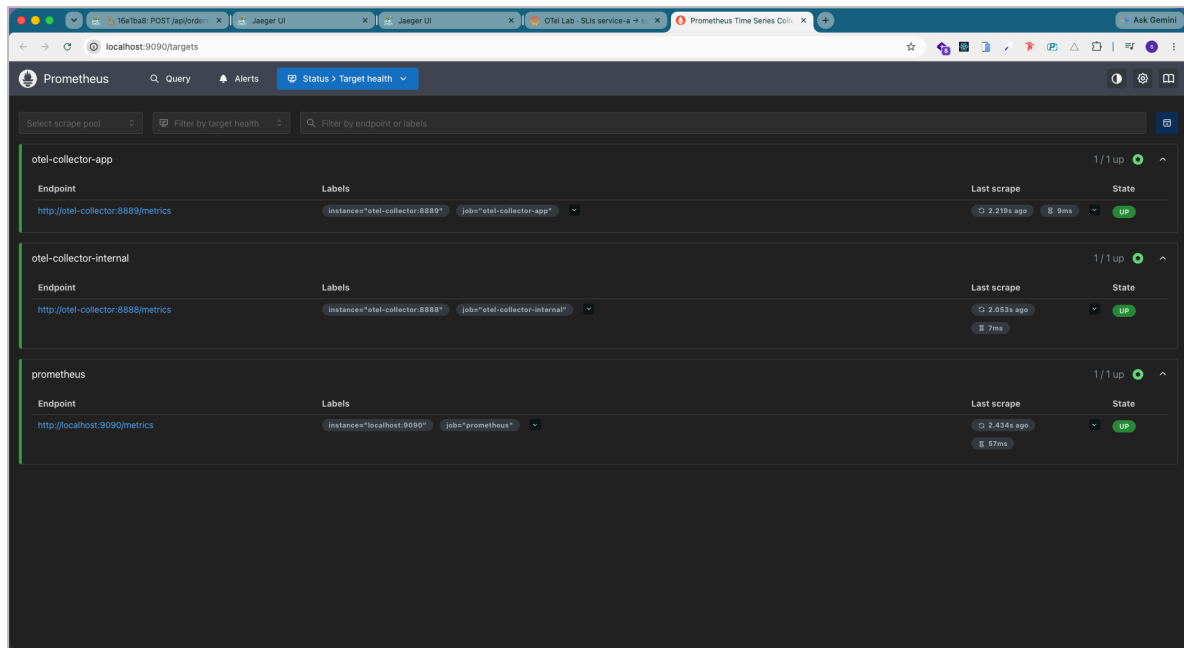


Figura 8

Prometheus como backend de métricas



Nota. Endpoints del Collector en estado UP: :8889 (métricas de la aplicación) y :8888 (métricas internas).

Figura 9

Consulta PromQL directa en Prometheus

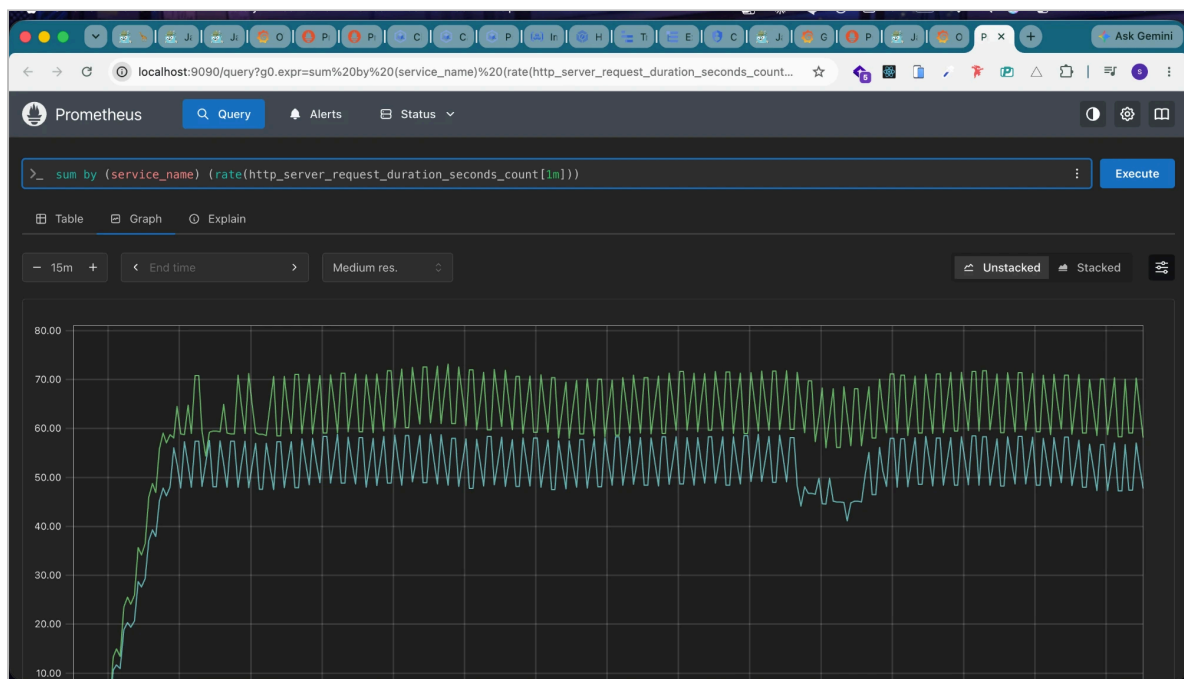


Figura 10

Logs de service-b en Loki con su trace_id

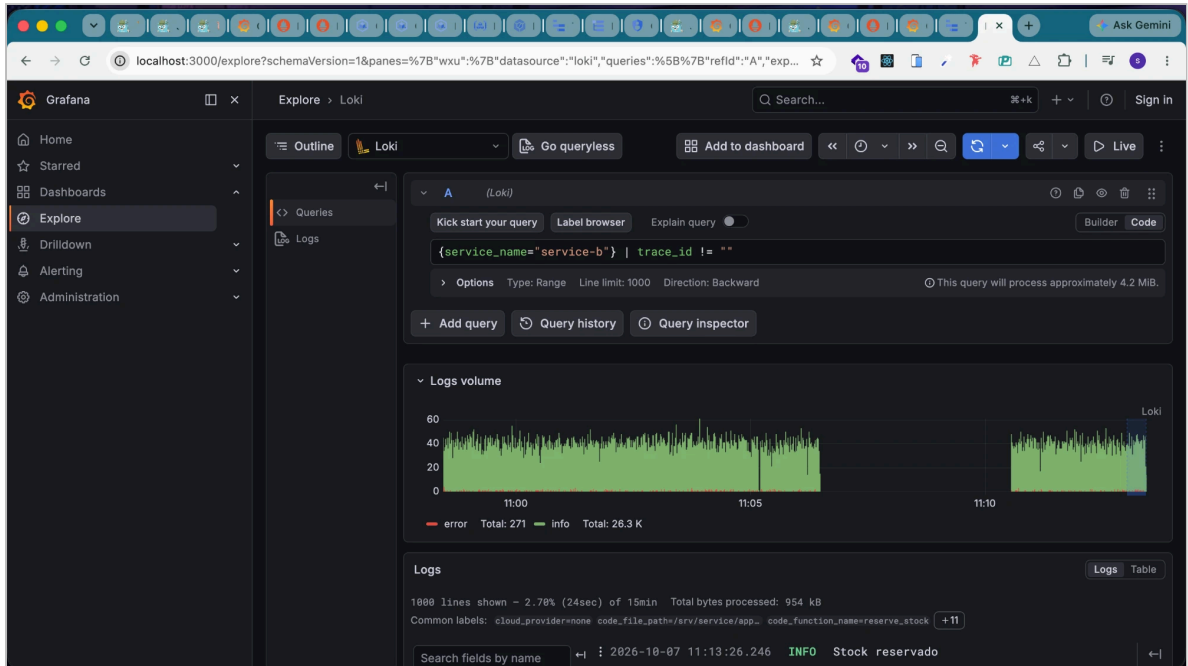
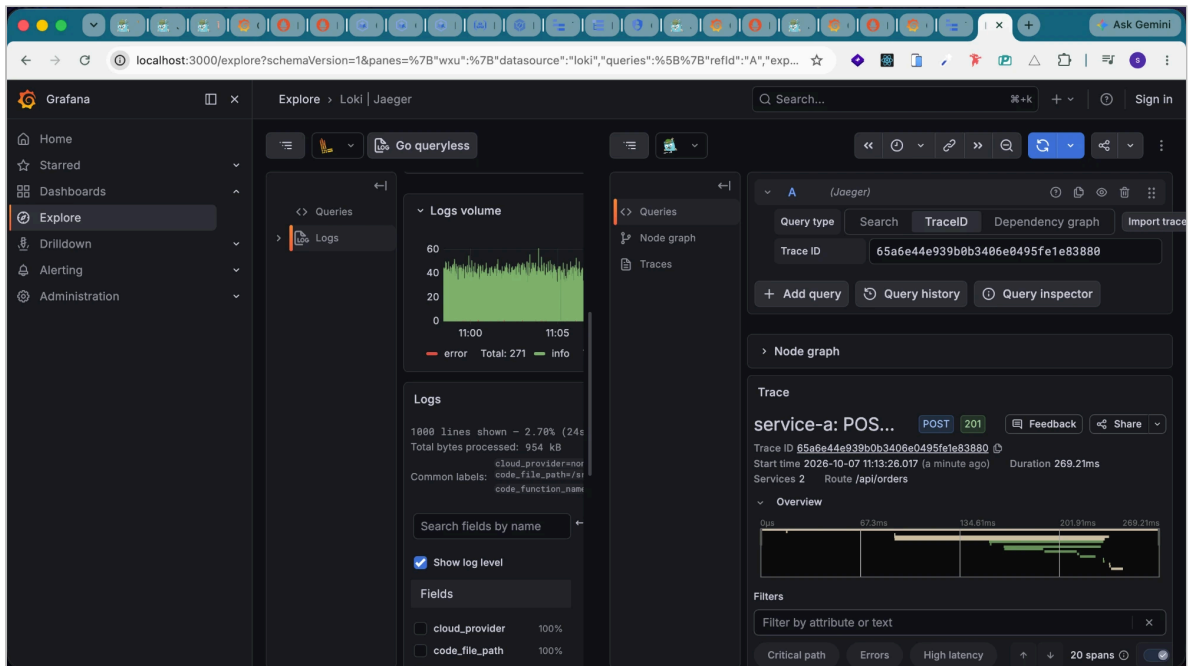


Figura 11

Correlación de log a traza en Grafana Explore



Nota. Desde una línea de log, el enlace "Ver traza en Jaeger" abre la traza del mismo trace_id en vista dividida.

Figura 12

Detalle de la traza abierta desde el log

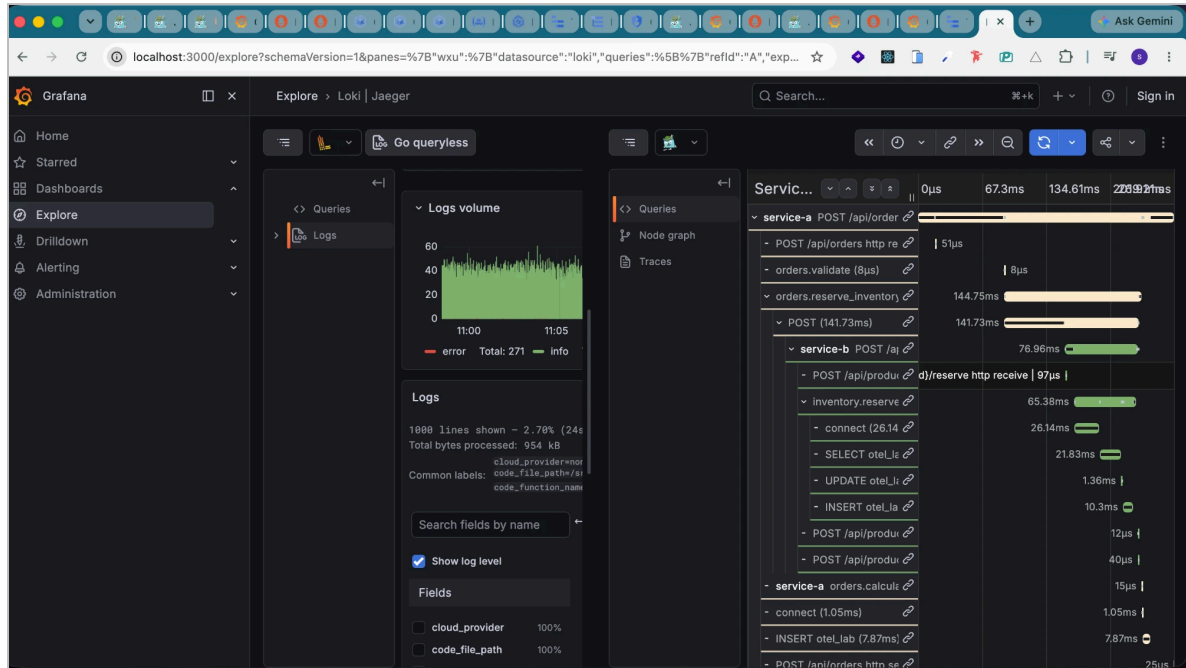


Figura 13

El mismo trace_id en los logs de ambos servicios

```
➤ OpenTelemetry git:(main) ✗ docker compose logs service-a service-b | grep 4e76d7a1c636f0c0f19c2a122ebba3ff
service-a-1 | {"timestamp": "2026-10-07T15:43:37.090699+00:00", "level": "INFO", "logger": "service-a", "message": "Orden creada", "service.name": "service-a", "trace_id": "4e76d7a1c636f0c0f19c2a122ebba3ff", "span_id": "d6eef3a83c60cc63", "trace_flags": "03", "order_id": "8e7ba26e-5fba-4d72-893b-a6ab262c0bf9", "customer_id": "c-evidencia", "product_id": 9, "total": 293.55}
service-b-1 | {"timestamp": "2026-10-07T15:43:37.035949+00:00", "level": "INFO", "logger": "service-b", "message": "Stock reservado", "service.name": "service-b", "trace_id": "4e76d7a1c636f0c0f19c2a122ebba3ff", "span_id": "3adf0475d4cc705b", "trace_flags": "03", "product_id": 9, "quantity": 12, "order_ref": "8e7ba26e-5fba-4d72-893b-a6ab262c0bf9", "remaining": 999990076}
➤ OpenTelemetry git:(main) ✗
```

Figura 14

Resultados del experimento A

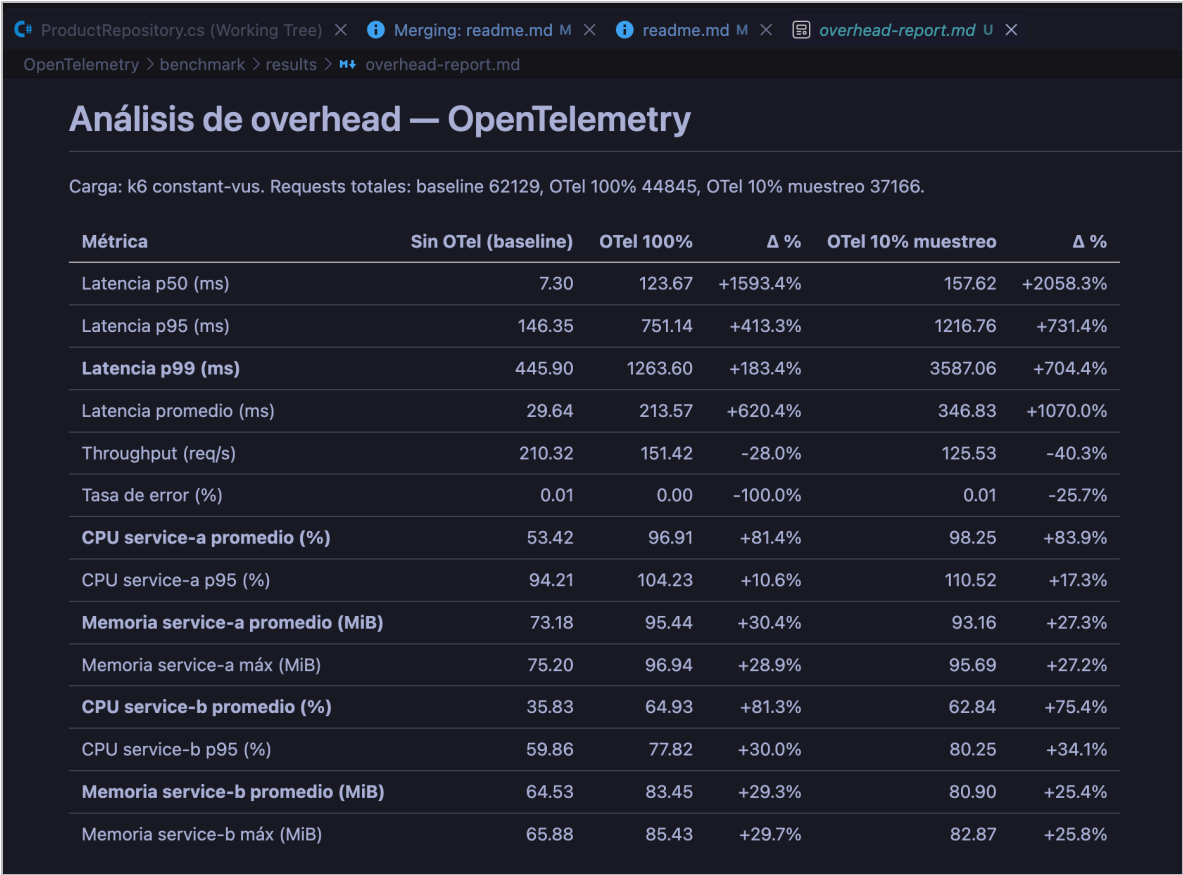


Figura 15

Resultados del experimento B

ProductRepository.cs (Working Tree)Merging: readme.md Mreadme.md Mablation-report.md UOpenTelemetry > benchmark > results > ablation > ablation-report.mdMarkdown Preview

Overhead por componente (ablación a tasa fija)

Carga: k6 constant-arrival-rate, misma tasa en todas las variantes (89.8 req/s medidos en baseline).

- CPU/req estable** = (mediana CPU service-a + mediana CPU service-b) / req/s, excluyendo los instantes con service-a saturado: es el costo intrínseco de la instrumentación.
- CPU/req promedio** usa la media de toda la corrida e incluye los episodios de saturación.
- % saturado** = muestras con service-a >= 90% de su core (episodios metaestables: cola + contención del GIL).

Variante	req/s	p50 (ms)	p99 (ms)	CPU a (%)	CPU b (%)	CPU/req estable (ms)	Δ vs baseline	CPU/req promedio (ms)	% saturado	Memoria a+b (MiB)
Sin OTel (SDK deshabilitado)	89.8	4.6	60.2	16.3	12.0	3.15	+0.00 ms (+0.0%)	3.30	0%	128.3
Instrumentación con API no-op (sin providers)	86.5	6.2	2120.6	22.1	16.1	4.42	+1.27 ms (+40.3%)	8.04	16%	159.1
Solo trazas (100%)	89.5	6.3	315.2	28.9	22.1	5.70	+2.55 ms (+81.2%)	6.33	2%	164.6
Solo métricas	90.0	5.5	380.3	21.7	15.9	4.18	+1.03 ms (+32.9%)	4.78	2%	157.2
Solo logs OTLP	89.6	5.4	90.4	20.6	15.5	4.03	+0.88 ms (+28.0%)	4.43	0%	152.2
Completo (trazas 100% + métricas + logs)	89.0	22.1	1718.7	41.7	31.3	8.21	+5.06 ms (+160.9%)	11.98	29%	172.8
Completo con muestreo 10%	89.5	7.1	937.0	30.4	22.3	5.89	+2.74 ms (+87.2%)	6.68	2%	163.9

CPU expresada como % de un core según `docker stats` (muestreo cada ~2 s); columnas CPU a/b = mediana en estado estable.

Google Cloud (GKE)

Figura 16

Clúster GKE creado con Terraform

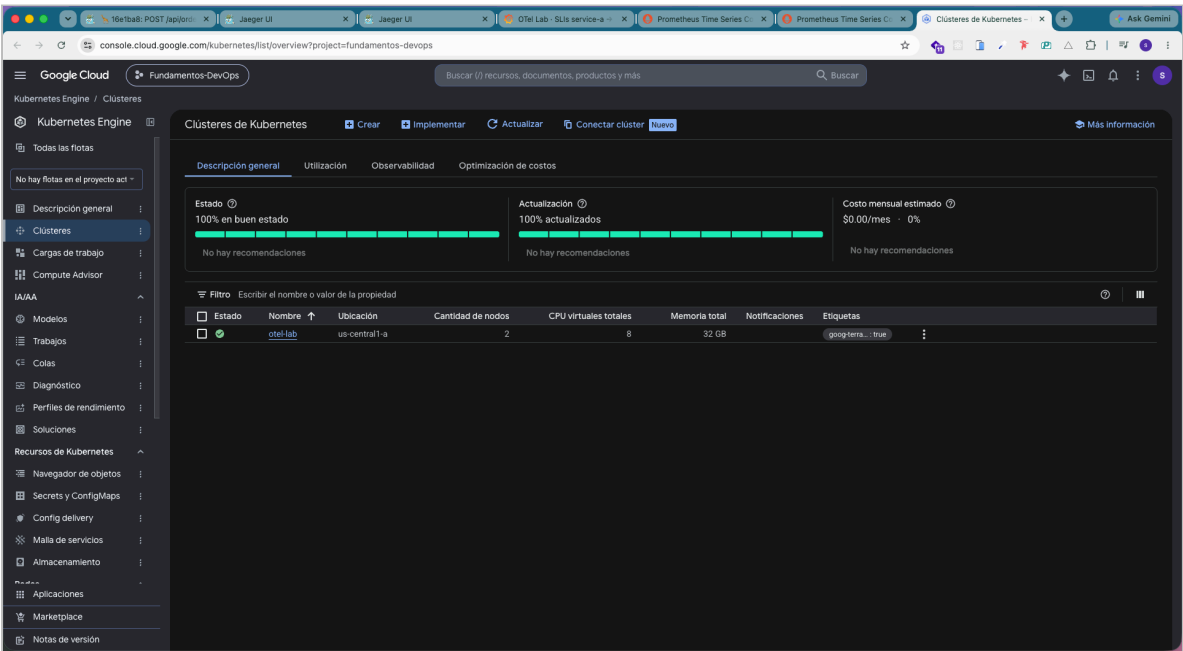


Figura 17

Cargas de trabajo en los namespaces otel-lab y monitoring

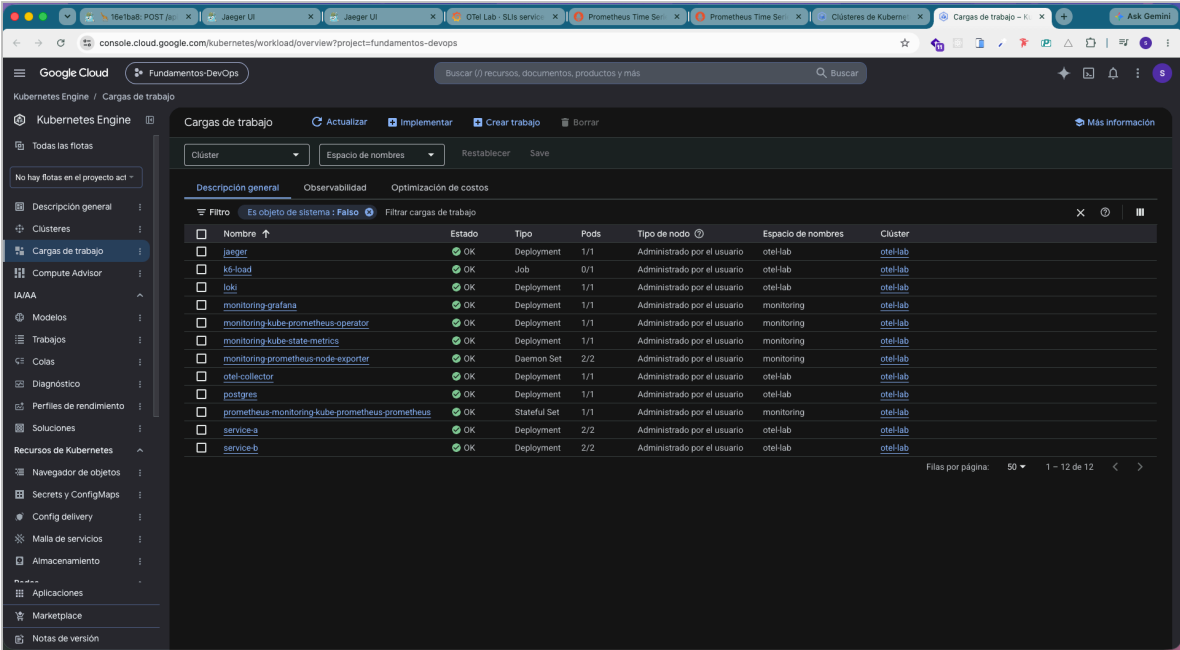


Figura 18

Servicios de Kubernetes y balanceador externo de service-a

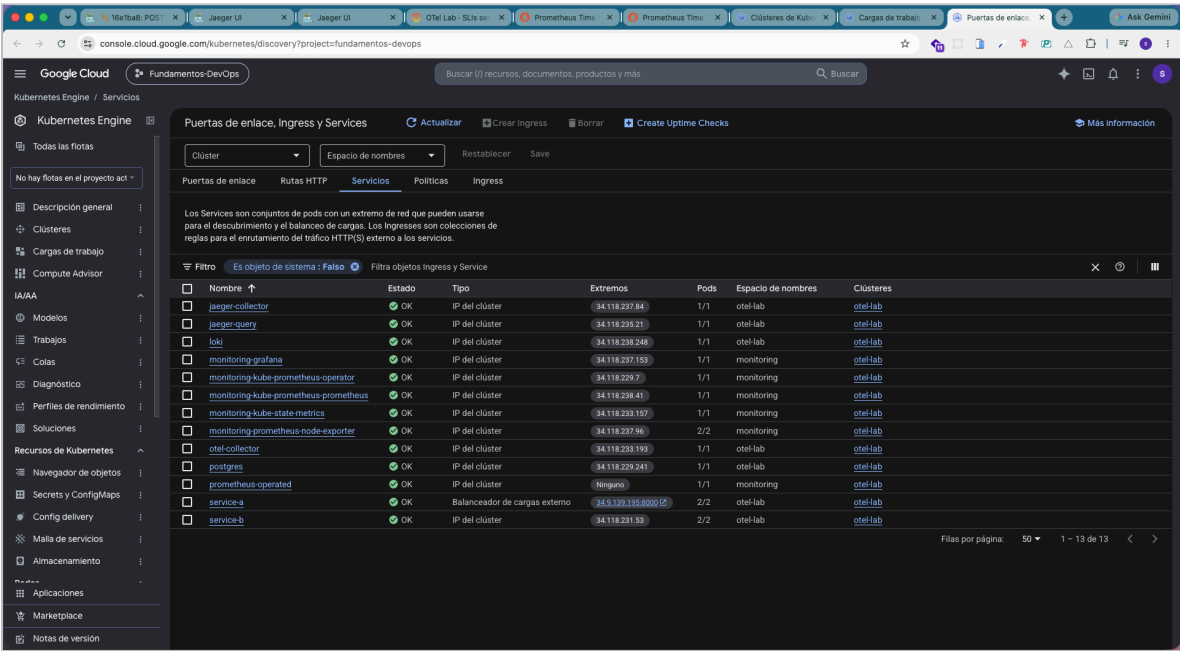


Figura 19

Imágenes publicadas en Artifact Registry

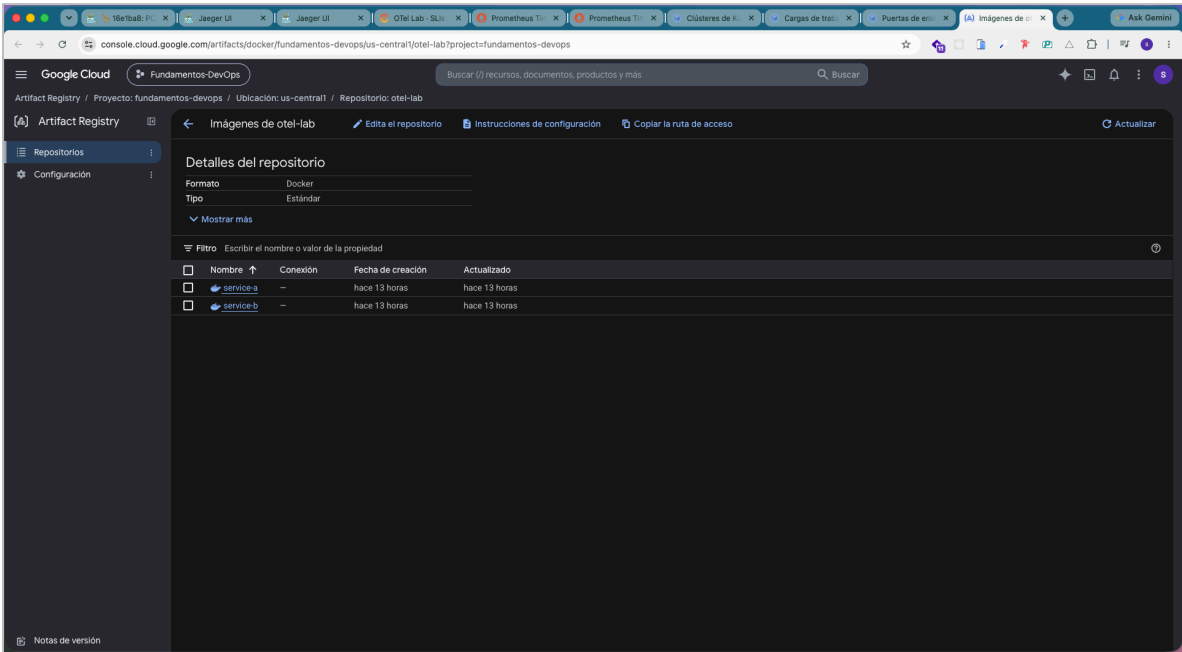


Figura 20

Construcción de imágenes en Cloud Build

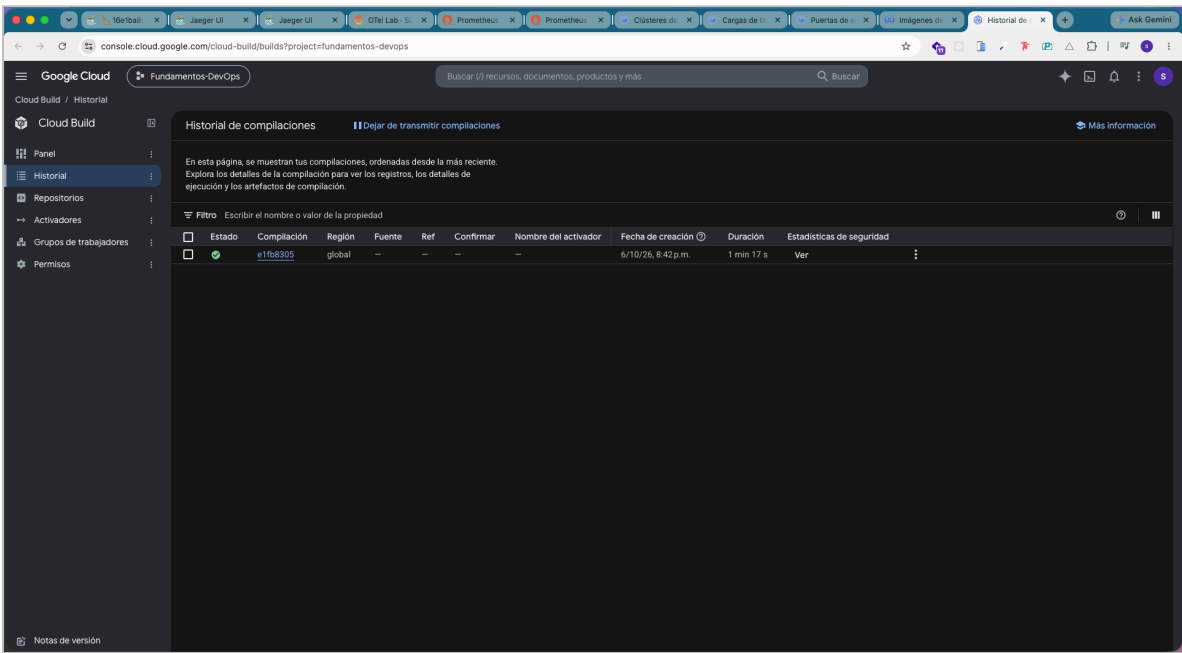
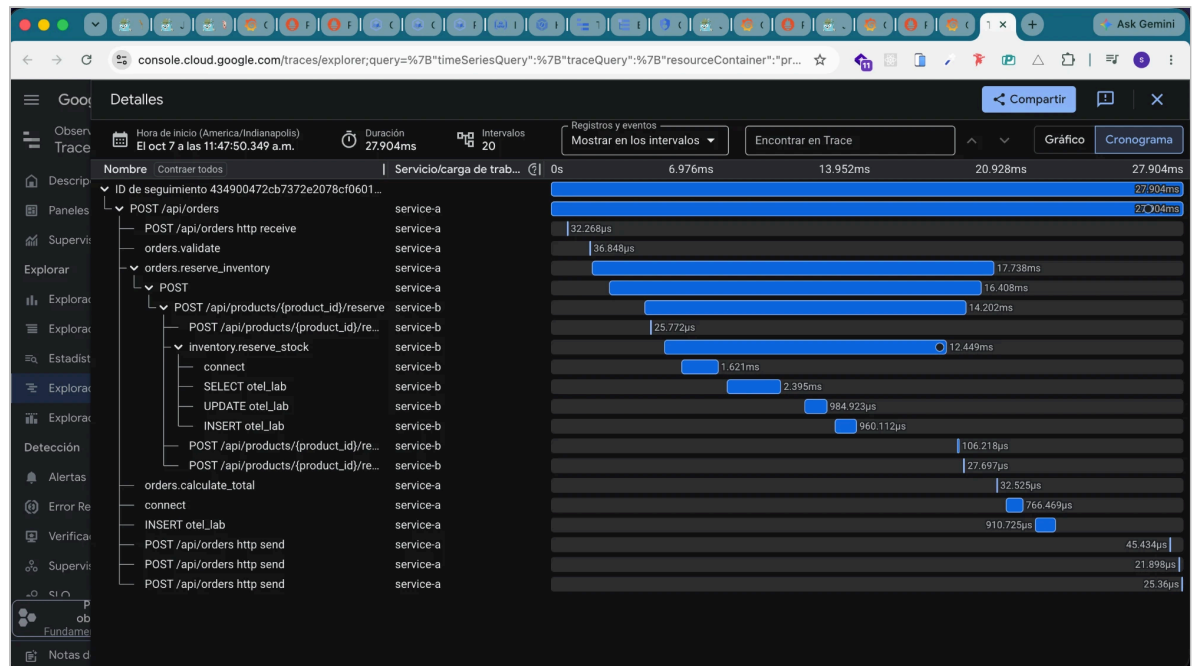
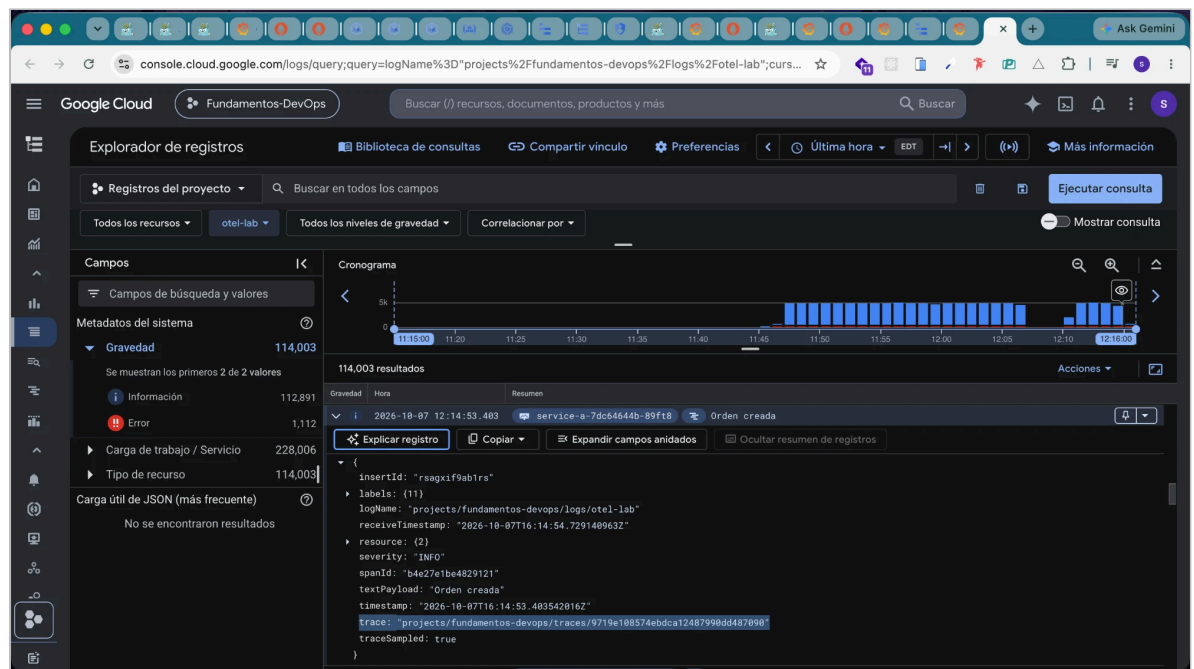


Figura 21
 Traza en Cloud Trace



Nota. Exportada por el Collector con el exporter `google-cloud`: 20 spans de service-a y service-b.

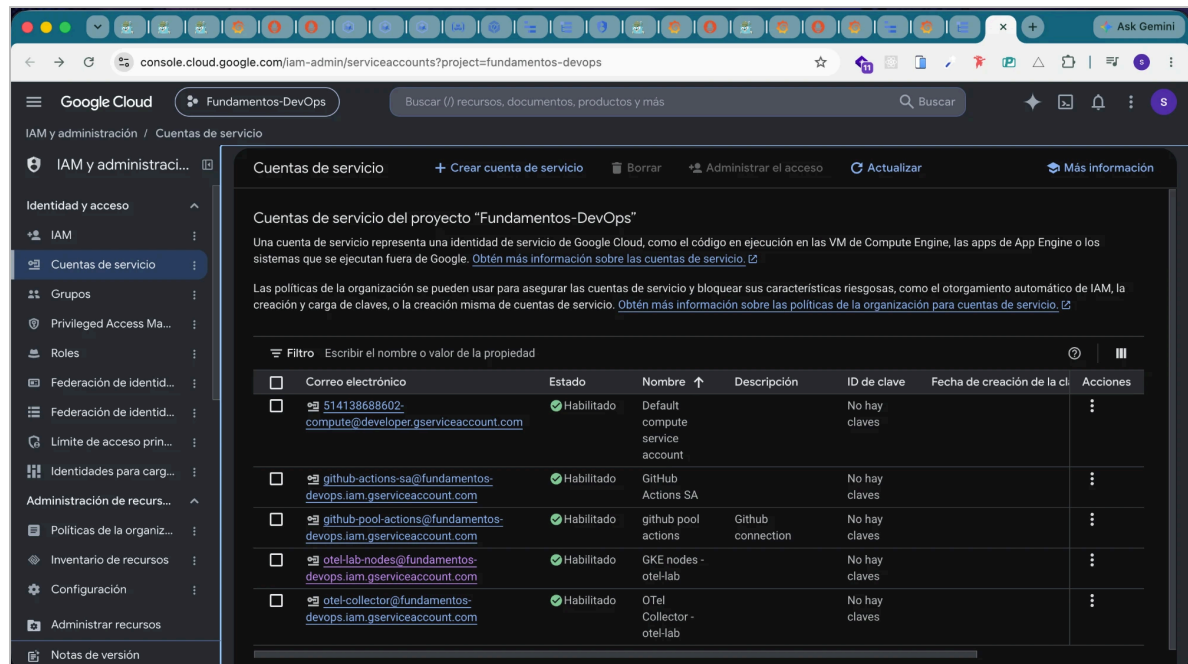
Figura 22
 Log en Cloud Logging vinculado a su traza



Nota. Los campos `trace` y `spanId` permiten la correlación nativa de log a traza en GCP.

Figura 23

Cuentas de servicio del laboratorio



Nota. otel-collector usa Workload Identity y no tiene llaves.

Figura 24

Traza en el Jaeger desplegado en GKE

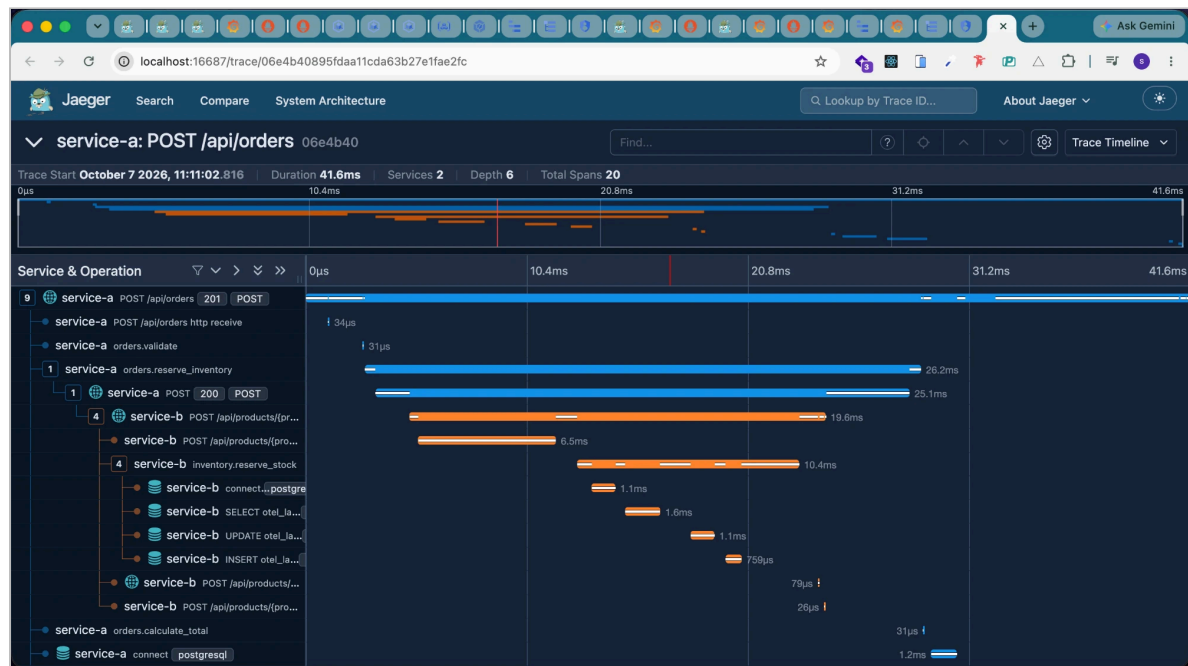


Figura 25

Dashboard en el Grafana de GKE

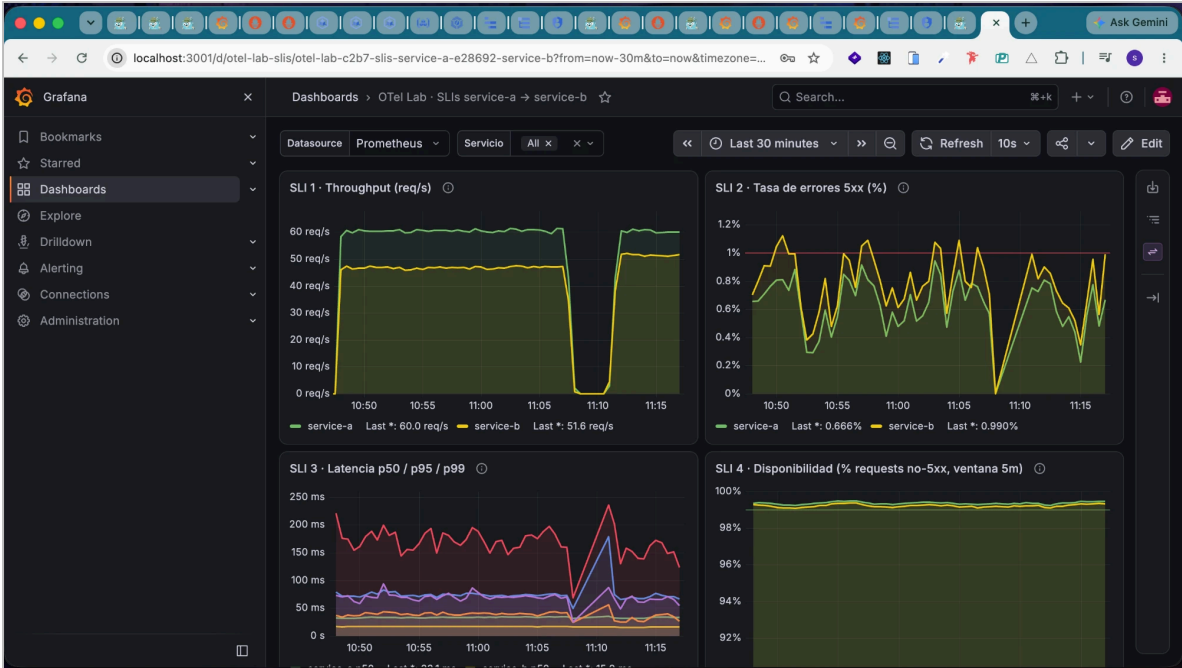


Figura 26

ServiceMonitor del Collector en el Prometheus de GKE

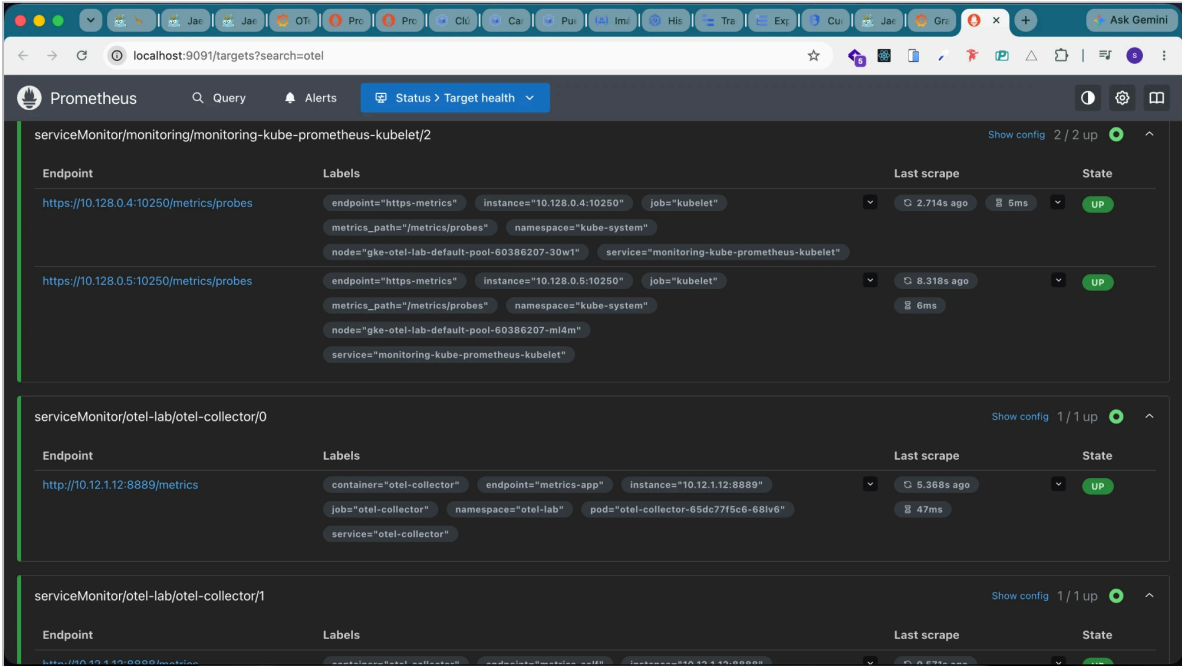


Figura 27

Pods y servicios del namespace otel-lab

Gk	Gk	Gk	Gk	Gk	k6	k6	x	v	+	:	↗	x	
NAME												READY	STAT
US		RESTARTS		AGE									
pod/jaeger-5d7cd97b5b-zxs2l												1/1	Runn
ing 0				106s									
pod/k6-load-hfbtj												1/1	Runn
ing 0				59s									
pod/loki-684b699888-8n6s5												1/1	Runn
ing 0				14h									
pod/otel-collector-65dc77f5c6-68lv6												1/1	Runn
ing 0				14h									
pod/postgres-dbd966bc8-2mbpn												1/1	Runn
ing 0				14h									
pod/service-a-7dc64644b-89ft8												1/1	Runn
ing 0				25m									
pod/service-a-7dc64644b-fm4z9												1/1	Runn
ing 0				26m									
pod/service-b-6dcf6cb65d-jtqk4												1/1	Runn
ing 0				25m									
pod/service-b-6dcf6cb65d-1ld7m												1/1	Runn
ing 0				26m									
NAME												TYPE	CLUSTER-
IP		EXTERNAL-IP		AGE		PORT(S)							
service/jaeger-collector						ClusterIP							34.118.2
37.84		<none>				4317/TCP							
				14h									
service/jaeger-query						ClusterIP							34.118.2
35.21		<none>				16686/TCP							
				14h									
service/loki						ClusterIP							34.118.2
38.248		<none>				3100/TCP							
				14h									
service/otel-collector						ClusterIP							34.118.2
33.193		<none>				4317/TCP, 4318/TCP, 8889/TCP							
				14h									
service/postgres						ClusterIP							34.118.2
29.241		<none>				5432/TCP							
				14h									
service/service-a						LoadBalancer							34.118.2
35.182		34.9.139.195				8000:30518/TCP							
				14h									
service/service-b						ClusterIP							34.118.2
31.53		<none>				8000/TCP							
				14h									
--- Recursos creados con Terraform ---													
google_artifact_registry_repository.otel_lab													
google_container_cluster.lab													

Figura 28

Recursos de GCP gestionados con Terraform

```
Gk Gk Gk Gk Gk k6 k6 x v + : ↵ x
14h
service/otel-collector      ClusterIP      34.118.2
33.193 <none>             4317/TCP,4318/TCP,8889/TCP
,8888/TCP 14h
service/postgres           ClusterIP      34.118.2
29.241 <none>             5432/TCP
14h
service/service-a          LoadBalancer   34.118.2
35.182 34.9.139.195      8000:30518/TCP
14h
service/service-b          ClusterIP      34.118.2
31.53 <none>             8000/TCP
14h

--- Recursos creados con Terraform ---
google_artifact_registry_repository.otel_lab
google_container_cluster.lab
google_container_node_pool.default
google_project_iam_member.nodes["roles/artifactreg
istry.reader"]
google_project_iam_member.nodes["roles/logging.log
Writer"]
google_project_iam_member.nodes["roles/monitoring.
metricWriter"]
google_project_iam_member.nodes["roles/monitoring.
viewer"]
google_project_iam_member.otel_collector["roles/cl
oudtrace.agent"]
google_project_iam_member.otel_collector["roles/lo
gging.logWriter"]
google_project_iam_member.otel_collector["roles/mo
nitoring.metricWriter"]
google_project_service.apis["artifactregistry.goog
leapis.com"]
google_project_service.apis["cloudbuild.googleapis
.com"]
google_project_service.apis["cloudtrace.googleapis
.com"]
google_project_service.apis["container.googleapis.
com"]
google_project_service.apis["logging.googleapis.co
m"]
google_project_service.apis["monitoring.googleapis
.com"]
google_service_account.nodes
google_service_account.otel_collector
google_service_account_iam_member.otel_collector_w
i
+ gcp git:(main) x
```

Figura 29

Resumen de k6 ejecutado como Job dentro de GKE

```
scenarios: (100.00%) 1 scenario, 30 max VUs, 15m10s max duration (incl. graceful stop):
  * steady_load: 30 looping VUs for 15m0s (gracefulStop: 10s)
[gke-evidencias] p99=120.3ms p95=73.8ms rps=61.4 errores=0.63%
```